Robotic Systems I
Escape Room Robot

Fotios Gkonos 1100849

Vasiliki Tsoulou 1101123

Chapter 1: Introduction and System Architecture

The objective of this project is the development and implementation of a fully autonomous navigation, mapping, and localization system for a mobile robotic platform that is designed to explore unknown environments, construct accurate 2D representations of the space, determine its precise location dynamically, and calculate optimal collision-free paths to designated target coordinates. Furthermore, the robot integrates computer vision capabilities to identify and interact with specific objects of interest, specifically ArUco markers that should then be used as keys to unlock restricted sections of the map. The system is built upon the Elephant Robotics MyAGV 2023 Pi, which is an omnidirectional mobile robot making it highly agile allowing for holonomic movement across the X and Y axes, as well as rotation around the Z axis. For environmental perception, the primary sensor utilized is a 2D LiDAR scanner which provides 360-degree range measurements, which serve as the foundational data for both mapping the environment's topology and correcting the robot's localization drift. The software framework powering the robot is ROS1, though some parts of the system are in ROS2, thus requiring a bridge between them. The system's architecture follows a modular, node-based design pattern, ensuring that independent processes can communicate efficiently via a publish-subscribe network. The modularity of this architecture ensures that the system is robust, easily debuggable, and scalable for future integrations. The basic structure of the system is shown in the diagram below (Figure 1).

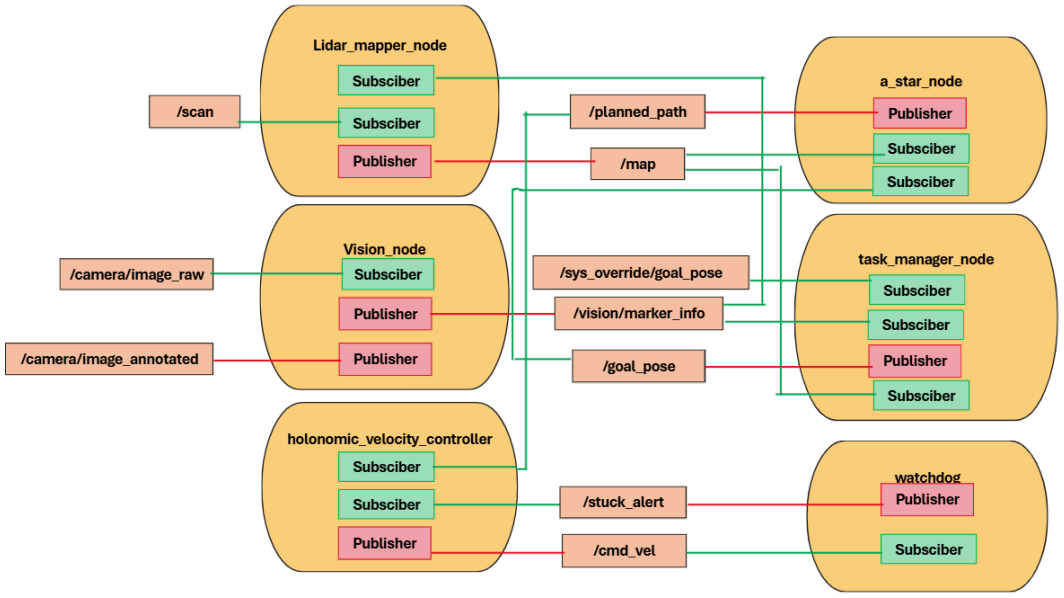


Figure 1:*Structural diagram of the entire system*

Chapter 2: Global Path Planning

For an autonomous mobile robot to navigate from a starting position to a target destination, it must first comprehend its environment as a traversable mathematical structure. In this system, the environment is represented as a 2D discrete Occupancy Grid. The grid divides the continuous physical space into a matrix of uniform cells, where each cell is assigned a specific probability or state of occupancy. In our implementation, the states are deterministic, free space is represented by the value 0, obstacles by 100 and unexplored areas by -1. Path planning is thus framed as a graph search problem. The robot acts as a node within this grid, and it must find a continuous sequence of adjacent free cells that connect its current location to the target coordinates without intersecting any obstacle cells. To solve the pathfinding problem efficiently, the A* Search Algorithm was implemented. To meet the real-time requirements of autonomous navigation and minimize computational overhead on the onboard Raspberry Pi, the standard A* algorithm was optimized into a Weighted A* implementation, where the algorithm evaluates each potential node n by calculating a total cost function f(n), defined by the following equation:

$$f(n) = g(n) + w * h(n)$$

Where g(n) is the exact accumulated cost of the path from the starting node to any given node n, h(n) is the heuristic estimated cost from node n to the final goal node, w is the heuristic weight and f(n) is the total estimated cost used to prioritize node expansion. By applying a weight greater than 1.0, the algorithm is heavily biased (greedy approach) toward expanding nodes that lead directly toward the goal. While Weighted A* sacrifices the strict guarantee of

finding the absolute shortest path, it drastically reduces the number of nodes explored and thus execution time, producing a suboptimal but collision-free path exceptionally quickly. The choice of the heuristic function h(n) dictates the behavior of the search and because the Elephant Robotics myAGV platform features omnidirectional wheels, it is capable of holonomic movement, meaning it can travel diagonally just as easily as it can travel along the cardinal axes. To leverage this kinematic capability, the search grid was configured for 8-way connectivity allowing horizontal, vertical, and diagonal movements. Consequently, the octile distance was selected as the heuristic function. The Octile distance heuristic is calculated as:

$$h(n) = max(dx, dy) + (\sqrt{2} - 1) * min(dx, dy)$$

This heuristic perfectly models the cost on an 8-connected grid, assuming a cost of 1 for orthogonal movements and approximately 1.414 for diagonal movements. Compared to the Euclidean distance, the Octile distance eliminates computationally expensive square-root operations while compared to the Manhattan distance, the Octile distance prevents the algorithm from generating artificial staircase patterns, resulting in smoother and more direct diagonal trajectories.

A critical requirement for the path planner is ensuring the robot's physical safety. Because the A* algorithm treats the robot as a dimensionless point -in this case a single grid cell- navigating strictly adjacent to an obstacle cell would result in a physical collision. This is mitigated by integrating an Obstacle Inflation mechanism (detailed in Chapter 3), which artificially expands the walls within the planner's memory, forcing the A* algorithm to route the path through the center of hallways and safely away from any walls. Once the algorithm successfully evaluates the goal node, the final phase is Path Extraction. The system traces backwards from the goal node to the start node using parent pointers assigned during the search phase. This sequence of nodes is translated from matrix indices back into real-world Cartesian coordinates and published as a standard ROS2 path message(under /planned_path), serving as the definitive blueprint for the robot's physical movement.

Before integrating the path planner with the live Lidar mapping node, the Weighted A* algorithm was rigorously tested using a static, simulated Occupancy Grid. This isolated testing environment allowed for the verification of the heuristic efficiency and the obstacle inflation logic (figure 2).

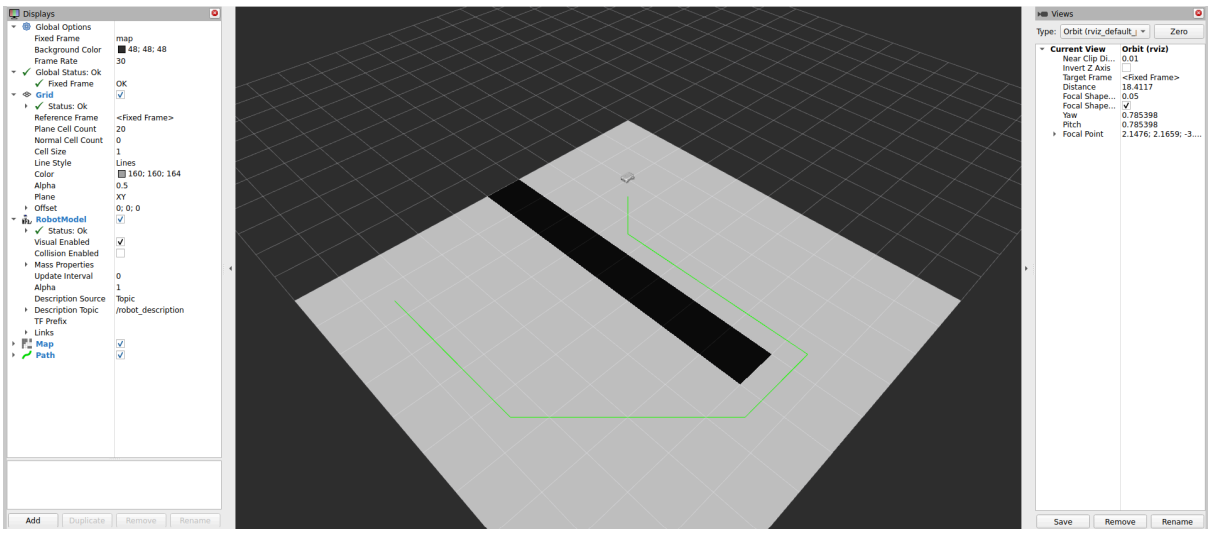


Figure 2: *Visualization of the Weighted A path planner navigating a mock Occupancy Grid within RViz. The generated path successfully routes the robot from the starting pose to the goal coordinates.*

Chapter 3: 2D Mapping and Environmental Reconstruction

To construct a spatial representation of the environment, the robot relies on raw planar data provided by the 2D LiDAR sensor. The sensor publishes a sensor_msgs/msg/LaserScan message under the /scan topic containing an array of range measurements that represents distances to obstacles at specific angular increments. These measurements are inherently in a Polar Coordinate System relative to the sensor's optical center. To be mathematically useful for mapping, these polar measurements must first be converted into Cartesian Coordinates within the robot's local frame which is set as base_footprint. For a given laser ray i with range r_i and angle θ_i, the local coordinates are calculated as:

$$x_{local\_i} = r_i * \cos(\theta_i)$$

$$y_{local\_i} = r_i * \sin(\theta_i)$$

Subsequently, these local coordinates must be transformed into the global map frame. Let the robot's current estimated pose in the global frame be defined by its translation (x_r, y_r) and its orientation θ_r. The global coordinates of the detected obstacle are derived using a standard 2D homogeneous transformation matrix:

$$\begin{bmatrix} x_{global\_i} \\ y_{global\_i} \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\theta_r) & -\sin(\theta_r) & x_r \\ \sin(\theta_r) & \cos(\theta_r) & y_r \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_{local\_i} \\ y_{local\_i} \\ 1 \end{bmatrix}$$

These continuous global coordinates (x_{global}, y_{global}) are then discretized by dividing them by the map's resolution which in this case equates to 0.05 meters/cell, to find the corresponding matrix indices within the Occupancy Grid. The next step is to update the Occupancy Grid which requires more than simply marking the obstacle's location. The fundamental premise of LiDAR mapping is that if a laser ray hits an obstacle at distance d, all space between the

sensor and distance d must be free of obstacles. This is achieved through a process called Raycasting. To determine exactly which grid cells the laser beam passes through, the custom mapper implements Bresenham's Line Algorithm. Bresenham's algorithm is highly optimal for grid-based operations because it relies entirely on integer addition, subtraction, and bit shifting, avoiding computationally expensive floating-point arithmetic. For each valid LiDAR range measurement Bresenham's algorithm calculates the sequence of discrete grid cells from the robot's center to the obstacle's coordinate assigning all intermediate cells along the ray an occupancy value of 0 (Free Space), effectively clearing out unexplored (-1) areas. The final target cell where the ray terminates is assigned a value of 100 (Obstacle), since this is where the ray stopped and thus represents a solid object. To ensure algorithmic robustness and prevent runtime crashes when laser rays extend beyond the predefined map boundaries, vectorized boolean indexing via the NumPy library was utilized. This safely filters out any calculated cell coordinates that exceed the matrix dimensions before the grid is updated.

A strict architectural rule enforced throughout the mapping and exploration process is the complete avoidance of nested Python for loops, which introduce severe computational bottlenecks. Instead, the occupancy grid is maintained purely as a 2D NumPy array, allowing for hardware-accelerated Vectorized Operations. This design choice transforms map-processing tasks that would normally cause severe latency in standard Python into instantaneous C-backend matrix calculations, ensuring the robot's perception loop runs at high frequencies.

While the raw Raycasting logic accurately maps the geometry of the room, it generates walls that are strictly one cell thick. As discussed in Chapter 2, navigating a dimensionless path planner adjacent to these walls would inevitably result in physical collisions due to the robot's actual chassis diameter. To bridge the gap between algorithmic planning and physical reality, Obstacle Inflation is applied to the map. A critical design choice in this architecture was to preserve the structural integrity of the base memory grid. Inflation is never applied recursively to the base map; instead, it is applied strictly to a copy of the grid immediately prior to publishing the map to the ROS2 network. To achieve high-performance inflation without iterating through the entire matrix in Python, Morphological Dilation from the scipy.ndimage and skimage.morphology libraries was employed. The inflation process utilizes a maximum filter with a strictly circular footprint : a structural disk where the disk radius is defined based on the robot's physical radius divided by the map resolution. For instance, a 0.15m robot radius -such as the one available- on a 0.05m/cell grid yields a disk radius of 3 cells. The maximum_filter evaluates the grid; if any cell within the disk's footprint contains a 100 (obstacle), the central cell is also raised to 100. Using a circular disk rather than a standard square filter ensures that the inflated obstacles feature perfectly rounded corners. This prevents the A* planner from cutting too sharply around wall edges, generating a safe, physically viable costmap that allows the robot to navigate complex environments with confidence. Furthermore, to handle dynamic environments and clear “ghost obstacles”, a custom clearing mechanism was introduced. When the LiDAR returns an infinite, invalid, or zero range reading indicating open space, the algorithm casts a virtual ray up to a predefined maximum clearing distance (3.5 meters). This

ensures that Bresenham's algorithm aggressively sweeps and marks all cells along this line of sight as free space, effectively removing transient noise without computationally overloading the system.

In addition, to ensure high-fidelity navigation, the mapping module was designed to support a dynamic, real-time refresh of the Occupancy Grid. Instead of relying on a static, pre-drawn map, the grid is actively updated at every LiDAR scan. This continuous raycasting allows the robot to instantly adapt to dynamic changes in the environment and because the map updates live, the robot constantly aligns its perceived location with the actual physical walls, eliminating some of the accumulation of odometry drift over time. However, this real-time updating introduced a specific edge case; the robot could occasionally become trapped by its own artificial obstacle inflation, causing the A* algorithm to fail. To resolve this, a dynamic footprint clearing mechanism was implemented. Immediately before the map is published to the ROS2 network, the algorithm forcefully clears the inflated cells in a strict radius surrounding the robot's current coordinates. This ensures the robot's center point is always classified as free space, guaranteeing that the A* planner can always successfully initialize a valid path sequence.

Chapter 4: Localization and Sensor Fusion

Accurate localization is the prerequisite for both mapping and path planning. If a robot cannot determine its precise coordinates within a global frame, any generated map will be distorted, and navigation will fail. The most fundamental method of localization relies on proprioceptive sensors, specifically wheel encoders, to calculate Wheel Odometry. By integrating the rotational velocity of the wheels over time, the system estimates the robot's displacement. However, the Elephant Robotics myAGV utilizes omnidirectional mecanum wheels. While these wheels provide exceptional holonomic maneuverability, they are inherently susceptible to mechanical slippage, friction inconsistencies, and uneven floor surfaces. Consequently, pure wheel odometry suffers from cumulative drift; minor angular errors rapidly compound over time, rendering the localization data unreliable over long distances. To correct the cumulative drift of wheel odometry, the system utilizes exteroceptive data from the LiDAR sensor. By comparing the current LiDAR scan with the previous scan, the robot can determine its precise relative movement by observing how the static environment appears to have shifted. In order to achieve this, a custom Iterative Closest Point (ICP) algorithm was developed in pure Python using NumPy vectorization. The goal of ICP is to find the optimal rigid transformation, comprising a rotation matrix R and a translation vector t, that aligns a source point cloud representing the current scan S, to a target point cloud that is gathered by the previous scan T. To ensure real-time performance and prevent CPU bottlenecks on the Raspberry Pi, the nearest neighbor search was heavily optimized using a KD-Tree data structure. Furthermore, an Outlier Rejection threshold was introduced; matched point pairs with a distance exceeding 0.3 meters are discarded. This prevents dynamic obstacles from corrupting the Singular Value Decomposition (SVD)

matrix(explained below in detail), ensuring the point cloud registration remains strictly reliant on static environmental features. Then, the optimal R and t are calculated to minimize the mean squared error between the matched point pairs and using these values, the source cloud S is transformed. This process repeats until the change in transformation falls below a defined tolerance threshold, signaling convergence.

To ensure computational speed and mathematical stability, the transformation estimation phase was implemented using Singular Value Decomposition (SVD) rather than gradient descent. For a set of matched point pairs $s_i \in S$ and $t_i \in T$, the centroids of both point clouds are first calculated:

$$\mu_s = \frac{1}{N}\sum_{i=1}^{N} s_i\,, \quad \mu_t = \frac{1}{N}\sum_{i=1}^{N} t_i$$

The points are then centered around the origin by subtracting their respective centroids:

$$s'_i = s_i - \mu_s, \quad t'_i = t_i - \mu_t$$

A cross-covariance matrix H is computed to capture the spatial correlation between the two centered point sets:

$$H = \sum_{i=1}^{N} (s'_i)\,(t'_i)^T$$

Applying Singular Value Decomposition to the covariance matrix yields $H = U * \Sigma * V^T$. The optimal rotation matrix R is then extracted as $R = V * U^T$. Once the optimal rotation is found, the translation vector t is simply the difference between the target centroid and the rotated source centroid:

$$t = \mu_t - R\mu_s$$

While the custom SVD-ICP algorithm is highly accurate, pure LiDAR localization suffers from a fatal geometric vulnerability known as the "Hallway Problem." In featureless environments such as a long, straight corridor, the LiDAR only perceives two parallel lines. If the robot moves forward, the scans look identical. A pure ICP algorithm will fail to detect longitudinal movement and incorrectly assume the robot is stationary. To build a truly robust system, a Loosely Coupled Sensor Fusion architecture was implemented, marrying the high-frequency but drifting wheel odometry with the low-frequency but highly accurate LiDAR ICP. Instead of treating the two systems independently, the kinematic displacement provided by the TF tree (wheel odometry) is injected into the ICP algorithm as an initial guess. Before the ICP correspondence search begins, the source point cloud is pre-transformed by this initial guess. Mathematically, this means the ICP algorithm no longer searches blindly for the robot's movement; it merely calculates the micro-corrections

necessary to fix the wheel slippage. This sensor fusion completely eliminates the Hallway Problem. In a featureless corridor, the ICP algorithm relies on the odometry's initial guess to maintain forward momentum, while simultaneously using the parallel walls to strictly correct any lateral or angular drift. The resulting architecture provides a highly robust, fully custom state estimation pipeline capable of navigating complex physical environments.

During the development of these computationally intensive algorithms such as the ICP state estimation and the A* path planner, the implementation of Just-In-Time (JIT) compilers, was rigorously evaluated. While JIT compilation can theoretically accelerate pure mathematics, it was consciously rejected for this architecture. Deploying Numba's LLVM compiler infrastructure on an ARM64 architecture (such as the Raspberry Pi) inside a Docker container introduces severe dependency instability. Furthermore, Numba cannot accelerate dynamic Python memory structures and provides zero performance gain over algorithms that are already heavily vectorized such as the SVD operations within the ICP, which directly call optimized Fortran/C libraries. Ultimately, the system achieves real-time execution speeds securely through strict algorithmic efficiency, Python's native heapq, and C-optimized spatial structures like KD-Trees.

A major challenge encountered during the live sensor fusion process was the map smearing effect. When the robot performs rapid rotational maneuvers, the mecanum wheels suffer from severe slippage, and the LiDAR sensor experiences angular distortion. This caused the ICP algorithm to occasionally calculate massive, false displacements, resulting in the entire Occupancy Grid jittering or shifting erratically in RViz. To counteract this, a smart update filtering mechanism, acting as an ICP sanity check, was integrated into the localization pipeline. The system continuously monitors the angular velocity and the discrepancy between the odometry's initial guess and the ICP's calculated output. If the discrepancy exceeds a safe physical threshold, the system temporarily rejects the ICP's output, relies strictly on the wheel odometry, and critically, pauses the live drawing of the map. By freezing the map updates during these volatile rotational spikes, the structural integrity of the generated map remains perfectly crisp, resuming live updates only when the robot stabilizes.

Chapter 5: Computer Vision and Semantic System Integration

While LiDAR-based localization and raycasting provide a robust geometric representation of the environment distinguishing purely between free space and physical obstacles, autonomous operations often require a higher level of spatial understanding. To solve the Escape Room navigation challenge, the robot must not only map the environment but also understand the semantic meaning of specific objects, namely, identifying keys to unlock corresponding doors. To achieve this, the system utilizes a monocular RGB camera and ArUco fiducial markers to dynamically alter the topological properties of the A* navigation grid. The vision subsystem operates within an independent ROS2 node

(vision_node). It subscribes to the robot's raw camera feed (/camera/image_raw) and utilizes the cv_bridge library to convert standard ROS image messages into multi-dimensional NumPy arrays compatible with OpenCV. The image is converted to grayscale, and the cv2.aruco.detectMarkers algorithm is applied using a predefined dictionary. When a marker is detected, the algorithm returns its unique ID and the sub-pixel coordinates of its four corners. The mathematical center of the marker in the image plane is calculated as the mean of these four corner coordinates.

To interact with the physical world, the 2D pixel coordinates of the detected marker must be translated into 3D spatial measurements. Rather than utilizing complex calibration matrices, the system employs the principles of the Pinhole Camera Model using similar triangles. By calculating the Euclidean distance between two adjacent corners of the detected marker, we obtain its perceived pixel width, W_{pixel}:

$$W_{pixel} = \sqrt{(u_1 - u_2)^2 + (v_1 - v_2)^2}$$

The physical distance Z in meters from the camera to the marker is then estimated as the physical width of the printed ArUco marker times the camera's approximated focal length be f measured in pixels divided by the perceived pixel width of the marker. Simultaneously, the angular offset φ of the marker relative to the camera's optical axis is approximated by calculating the displacement of the marker's center u_c from the image's principal point which is the horizontal center of the frame c_x.

$$\varphi = (c_x - u_c) * k$$

Where k is a camera-specific constant mapping pixels to radians. The vision_node packages these three variables—ID, Distance (Z), and Angle (φ)—into a Float32MultiArray message and publishes it to the ROS2 network, acting as a lightweight, custom semantic sensor.

The core intelligence of the Semantic architecture resides within the Mapper Node. Upon receiving the spatial data from the camera, the node projects the marker's local coordinates into the global map frame using the robot's highly accurate ICP-corrected pose (x_r, y_r, θ_r). The absolute global angle of the marker is calculated as:

$$\theta_{global} = \theta_r + \varphi$$

The global Cartesian coordinates (x_m, y_m) of the marker are then derived using standard trigonometry:

$$x_m = x_r + Z * \cos(\theta_{global})$$

$$y_m = y_r + Z * \sin(\theta_{global})$$

These continuous physical coordinates are discretized by dividing them by the map's resolution to locate the exact cells the object occupies within the Occupancy Grid.

To process the logic of the Escape Room, the system utilizes a strict ID classification rule where IDs < 100 are classified as keys and IDs ≥ 100 are classified as doors. The mapper node maintains three dynamic memory structures, a set of found_keys, a dictionary mapping door IDs to their physical grid cells as known_doors, and a set of unlocked_doors. The system performs continuous bidirectional checks where if a newly detected key corresponds to a known door, or if a newly discovered door corresponds to a previously stored key, an unlock event is triggered.

The final and most critical challenge of this architecture is integrating the semantic data with the physical LiDAR data. In a standard setup, even if the vision system logically unlocks a door, the LiDAR will continue to perceive the physical marker as a solid obstacle, forcing the raycasting algorithm to draw a wall over the door's coordinates, thereby blocking the A* planner. To resolve this conflict, the Bresenham raycasting function was fundamentally modified to accept the unlocked_cells_set as an overriding parameter. During the map update cycle, when a laser ray terminates at a specific cell, the algorithm checks the semantic memory before updating the grid. If the cell is in the unlocked_cells_set, the physical obstacle is intentionally ignored, and the cell is forcefully set to 0. Otherwise, it is recorded normally as 100. This custom integration creates a highly dynamic and responsive system where the A* path planner remains purely mathematical and agnostic to the concept of "doors" or "keys". The moment the camera visually verifies a key, the mapper seamlessly drops the corresponding walls from the grid memory. On the very next computational cycle, the A* algorithm instantly detects the newly opened topological pathway and routes the robot through the physical doorway, achieving full systemic autonomy.

Chapter 6 :High-level safety and monitoring of the robot

One important factor that must be considered is the behaviour of the robot during failed movement attempts, i.e., when the robot is commanded to move but does not achieve meaningful displacement in the environment or during wheel slippage. It is essential that the robot can detect and recover from such situations autonomously.

In this implementation, the watchdog node is responsible for detecting such conditions and publishing a Boolean alert on the /stuck_alert topic, allowing other components of the holonomic_velocity_controller to initiate a recovery behavior. The node subscribes to the /cmd_vel topic to determine whether motion is being requested. Every time a new velocity command is received, the node extracts the commanded linear and angular velocities. The linear speed is calculated as the Euclidean magnitude of the velocity components in the x and y directions, while the angular speed is obtained from the absolute value of the rotational velocity around the z-axis. These values are stored and later used to determine whether the robot is expected to be moving.

TF transformations (map to base_footprint) are used to estimate the robot’s real-world displacement. If the transformation is unavailable due to communication delays or missing data, the node safely skips the current evaluation instead of generating an incorrect alert.

The core functionality of the node is implemented in the check_stuck() function, which is executed once every second using a ROS timer. The function first checks whether the node is currently in a cooldown period. After a stuck condition has been detected, a five-second cooldown is enforced to prevent repeated alerts while another recovery mechanism attempts to free the robot. During this period, the node simply publishes False on the /stuck_alert topic.

If the cooldown has expired, the node verifies that the robot is actually commanded to move. When the commanded linear velocity is below 0.05 m/s, the robot is considered stationary by intention, and no stuck detection is performed. Similarly, if the robot is commanded to rotate with an angular velocity greater than 0.1 rad/s, the evaluation is skipped because a robot rotating in place naturally exhibits very little translational movement, which could otherwise result in false stuck detections.

When the robot is expected to move forward, the node retrieves its current position and stores it in a history buffer together with the current timestamp. The history maintains only the most recent four seconds of position data, ensuring that outdated information is discarded. Before making any decision, the algorithm verifies that at least three seconds of position history are available. This prevents the node from drawing conclusions based on insufficient data immediately after startup.

If motion commands are present but the robot's displacement remains below a predefined threshold (approximately 0.05 m over 3 seconds), the system classifies this as a potential stuck condition. This situation may arise due to obstacles, wheel slippage, or collisions. In response, the node publishes a Boolean alert on the /stuck_alert topic, which can be used by higher-level control components to trigger recovery behaviours, it clears the stored position history, and starts the cooldown timer.

If the robot has moved more than the minimum distance threshold, the node determines that the robot is operating normally and publishes False on the /stuck_alert topic. The monitoring process then continues, allowing the node to continuously evaluate the robot's motion throughout its operation.

Chapter 7 :Velocity control

One crucial part of this project is managing the velocity commands for the robot's wheels. The holonomic_velocity_node implements a velocity controller based on a PI control strategy with an additional recovery state machine for stuck detection. Its main role is to follow along the path provided by the a start algorithm and provide continuous velocity commands (/cmd_vel) that drive a holonomic mobile robot in the map frame.

The choice of a PI instead of a PID or a PD controller is intentional. The PI provides an effective balance between tracking accuracy, implementation simplicity, and robustness for the myAGV mobile robot and it enhances system stability by eliminating steady error. One large problem is the jittering because of the localization data obtained through TF (which

is inherently noisy), so the derivative term was intentionally omitted since derivative control amplifies measurement noise, introducing oscillations in the commanded velocities. Because the robot operates at relatively low speeds, the PI controller was sufficient to maintain a smooth and stable path-following performance. Additionally, the lookahead-based path tracking strategy and heading correction logic already reduce overshoot, making the derivative term unnecessary for this application.

A significant feature of this node is its recovery state machine, designed to handle situations where the robot becomes stuck. When a true message from the topic /stuck_alert is received, the controller transitions from **NORMAL** mode into a two-stage recovery behavior. First, in the **BACKING_UP** state, the robot moves backward at a fixed low velocity for a short duration, attempting to disengage from obstacles or deadlocks. Then, in the **SPINNING** state, it rotates in place to reorient itself and increase the likelihood of finding a free path. After completing these recovery actions, the controller returns to **NORMAL** operation and resumes tracking the original goal. This design ensures that transient failures in navigation do not permanently interrupt motion, while still preserving the integrity of the higher-level planning system.

Under the **NORMAL** state, the PI controller runs in a continuous loop. Firstly, the controller takes the measurements of the /map topic for the robot's current position. It then checks if this distance from the final position falls below the predefined positional tolerance. If the goal position isn't reached,the robot needs to follow the sequence of waypoints set from the a star algorithm, received under the /planned_path topic.

In order to follow the path, it first looks for the nearest waypoint. Waypoints that have already been passed are removed from the stored path to prevent the controller from attempting to navigate towards previously visited locations. Instead of steering directly toward the nearest waypoint, the controller employs a lookahead strategy inspired by the Pure Pursuit algorithm. A target point is selected at a predefined distance ahead of the robot along the planned path. This lookahead point acts as the immediate navigation objective, allowing the robot to generate smoother trajectories and avoid abrupt changes in direction. As the robot approaches the final destination, the lookahead point automatically becomes the goal itself, enabling accurate stopping at the target position.

In order to generate the motion commands for the wheels, firstly the calculation of the global position error (error_x_global, error_y_global) and the heading error (error_theta) relative to the target pose are necessary. The angular error is normalized using atan2(sin, cos) to ensure smooth wrap-around between angles, preventing discontinuities at $\pm\pi$. However, to properly control a holonomic platform, the global position errors are transformed into the robot's local coordinate frame. This is done using a standard rotation matrix based on the robot's current yaw. The resulting local errors (error_x_local, error_y_local) represent how far the robot should move forward and sideways relative to its own orientation. After testing, the robot proved not to be fully omnidirectional, so solely the velocity in the x axis was calculated by multiplying the local error to the proportional gain and adding the product of the integral with the integral gain.

Angular velocity is computed similarly using a second proportional and integral controller. This separation allows the robot to simultaneously translate in any direction while also rotating toward the desired orientation.

The controller also includes saturation limits to ensure safe and stable motion. The combined linear velocity vector is clamped to a maximum magnitude (max_linear_vel), preventing excessive speed during large position errors. Similarly, angular velocity is constrained using np.clip to avoid unstable or overly aggressive rotation.

Chapter 8 :Frontier Base Exploration

The core mechanism for autonomous mapping during the initial exploration phase in this project is frontier-based exploration. It is implemented in the task_manager_node, specifically in the state as **EXPLORE_FOR_MARKERS**. Frontier exploration is a common strategy in robotics where the robot systematically moves toward the boundary between known free space and unknown space in an occupancy grid map, gradually expanding its knowledge of the environment.

Frontier cells are defined as free cells that are adjacent to at least one unknown cell. These represent the “edge” of explored space and are the most informative locations for continued exploration. The find_closest_frontier() function is responsible for selecting the next exploration target by identifying the nearest reachable frontier cell on the occupancy grid. After generating all candidate frontier cells, the function performs a flood-fill algorithm implemented using Breadth-First Search (BFS) starting from the robot's current position to determine which free cells are actually reachable. The search expands through all connected free cells using 8-way connectivity, allowing both orthogonal and diagonal movement. By performing a logical AND operation between the detected frontier cells and the reachable cells, the algorithm eliminates inaccessible or "ghost" frontiers that are separated from the robot by obstacles. The reachability analysis prevents the robot from selecting frontier cells that are geometrically adjacent to unexplored space but are separated by walls or obstacles. Without this filtering stage, the planner could repeatedly select unreachable exploration targets, leading to unnecessary replanning and inefficient exploration. The remaining valid frontier cells are converted from grid indices into world coordinates.

The function then removes frontiers that are too close to the robot, preventing the selection of targets that have effectively already been reached. In addition, previously failed exploration targets stored in a blacklist are excluded by rejecting all frontiers within a predefined radius of those locations, preventing the robot from repeatedly attempting unreachable goals. Finally, the Euclidean distance between the robot and each remaining frontier is calculated, and the closest valid frontier is selected and returned as the next exploration goal. This greedy selection strategy ensures efficient exploration by always moving toward the nearest unknown boundary, reducing redundant travel and accelerating map coverage. If no suitable frontier remains after any of the filtering stages, the function returns none, indicating that no valid exploration target is currently available. Resultingly, the

system concludes that initial exploration is complete and transitions to higher-level behaviors such as door navigation.

Once a frontier cell is detected, it is converted from grid coordinates into real-world coordinates using the map origin (origin_x, origin_y) and resolution (map_resolution) and then published under the /goal_pose topic for the a_star algorithm to use.

Chapter 9:Task-level decision making

At a high level, the robot is required to execute a sequence of tasks in order to autonomously explore an unknown environment. Initially, it explores all reachable frontier regions of the map while identifying visual markers corresponding to keys and doors. The detected keys are stored and associated with specific doors according to a predefined key-door mapping. Once the initial exploration phase is complete, the robot navigates only to those doors for which the corresponding key has been discovered, enabling exploration of previously inaccessible regions. After all accessible doors have been investigated and the environment has been fully explored, the robot returns to its starting position. To coordinate these activities, a hierarchical task manager (task_manager_node) is implemented. The node operates as a finite state machine (FSM), managing the robot's behaviour according to information received from the mapping, localization, and vision subsystems. A finite state machine was selected because the robot's mission naturally consists of distinct operating modes with clearly defined transitions. This architecture simplifies decision making, improves modularity, and ensures that only one high-level behaviour is active at any given time.

At startup, the node initializes in the **EXPLORE_FOR_MARKERS** state, where the robot autonomously explores the environment using a frontier-based exploration strategy (described in chapter 8). By navigating toward the closest frontier, the robot incrementally expands the known map while searching for visual markers. Once a frontier has been reached or the frontier times out, the next one is set. The exploration halts when there aren’t any valid frontiers left.

Marker detection is provided by the vision subsystem through the /vision/marker_info topic. The Task Manager categorizes markers into two groups: keys (marker IDs below 100) and doors (marker IDs equal to or greater than 100). Detected keys are stored in a set to prevent duplicate registrations, while detected doors are stored together with their estimated locations. The detected key identifiers are subsequently used to determine which discovered doors may be investigated. This introduces a dependency between the exploration and door-investigation phases, ensuring that the robot only attempts to access regions for which the required key has been acquired.

During the **NAVIGATE_TO_DOOR** state, the node considers only unexplored doors whose corresponding keys have already been detected. Using the predefined key-door association, inaccessible doors are filtered out from the candidate set. Among the remaining valid doors, the robot selects the nearest target according to the Euclidean distance between its current position and the recorded door location. A navigation goal corresponding to the

selected door is then issued to the navigation subsystem, after which the state machine transitions to **EXPLORE_DOOR**. When all known doors have been investigated, the robot switches to the **RETURN_HOME** state.

In the **EXPLORE_DOOR** state, the robot performs localized frontier exploration within the area behind the selected door. This allows the system to map newly accessible regions and discover additional markers. To ensure that the robot explores the newly accessible room rather than selecting an arbitrary frontier elsewhere in the map, frontier selection is spatially constrained using a circular search region centred on the robot. Only frontiers located within this radius are considered valid exploration targets. Once no further frontiers are available, the area is considered fully explored and the corresponding door is added to the explored door list. The Task Manager then returns to the door selection state and repeats the process for any remaining unexplored doors.

In the **RETURN_HOME** state, a navigation goal corresponding to the predefined home position is issued and monitored until the robot successfully reaches the starting location. Upon successful return, the mission is declared complete and the task manager ceases operation.

In addition to the autonomous operation, the node supports a manual override mechanism through the /sys_override/goal_pose topic. When an override command is received, the current autonomous state is temporarily suspended and stored. The robot immediately redirects to the operator-specified target location. Once the override objective has been reached, the Task Manager restores the previously active state and resumes autonomous execution without losing mission progress. The same functionality may also be invoked directly through RViz by specifying a 2D Goal Pose, providing a convenient interface for testing and manual intervention during development.

Overall, the Task Manager Node provides centralized mission coordination by integrating mapping, localization, vision-based perception, and navigation. Through its finite state machine architecture, it enables autonomous exploration, target discovery, room investigation, mission completion, and operator intervention within a single coherent decision-making framework. It's important to note that the Task Manager maintains internal data structures that preserve the discovered keys, detected door locations, explored doors, and previously failed navigation goals throughout the mission. This persistent memory enables informed decision making in subsequent states without requiring repeated exploration of previously visited areas.

Chapter 10:Setup Procedure and challenges

The first actual task of the project setup consists of gathering the information from the robot's sensors, specifically from the odometry and the lidar. The nodes of the mobile robot myAGV (that are used in this project) run on ROS. However, the visualization in rviz as well as all of our custom nodes are in ROS2. This calls for a ros_bridge, linking both of them together and passing the /odom and /scan topics from ROS over to ROS2. We needed to

modify the bridge.yaml to include the desired topics that needed to be passed on from one version to another. Also, we needed to use the built-in drivers for the LiDAR and the odometry.

First of all, some of the nodes were individually tested. Specifically, we first tried the vision_node, involving the camera and the detection of AruCo markers which seemed to perfectly function. Then we moved on to the lidar_mapper node where we came across the following issues.

Initially, we encountered issues with the rotation of the LiDAR. To counter this, we needed to rotate by 180 degrees the received measurements. The maximum distance of the sensor was experimentally chosen to be 3.5 meters as a common ground between scanning the whole map and minimizing the effect of random noise. Another issue that occurred, was the time mismatch between the LiDAR measurements and the TF request of them. This shut down our lidar_mapper_node as it was requesting for measurements that were yet to appear. To counter this, we just use the current position of the wheels, bypassing the delay of the bridge.

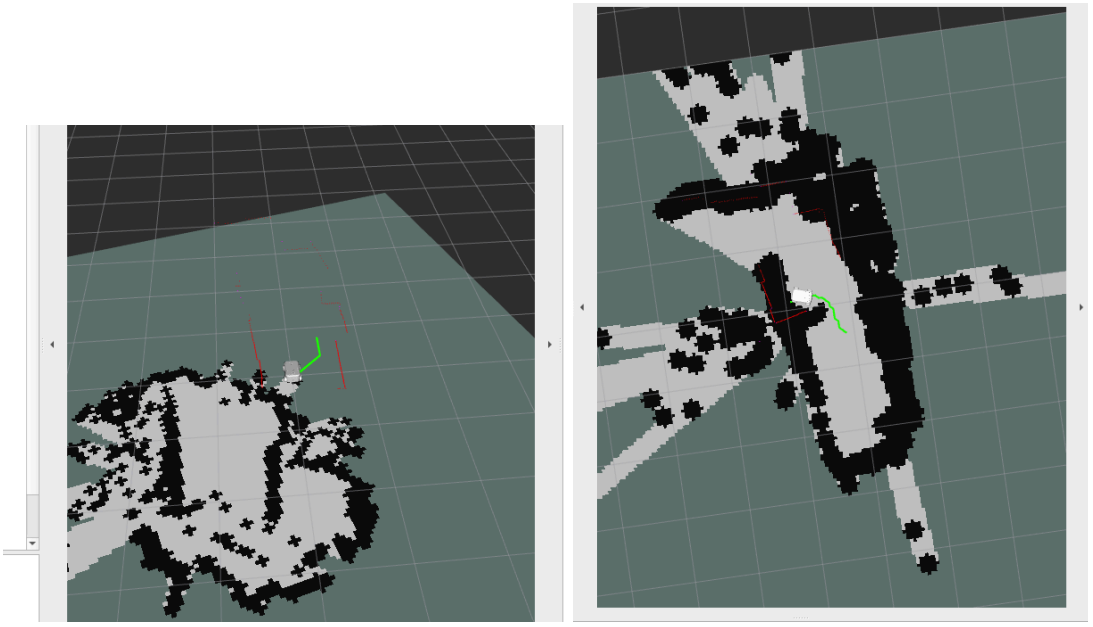
In order to both test our mapping and the robot's functionality we used the pre-build teleop function for the robot. This brought to our attention more issues with the mapping, specifically while it was stationary the map was jittering and this effect worsened by the motion of the robot, especially at higher speeds. The changes necessary for the solution of this problem were made to the ICP file, where we were previously using numpy broadcasting, but for this application it proved to be costly. Instead of that, the KD-tree structure was used which searches nearest points in logarithmic time. In addition, there was a need for an outlier rejection part to be added to the algorithm, since it always tried to match nearest points regardless of the distance between them, polluting our data with faulty measurements. Initial testing revealed that the Occupancy Grid was static and not reflecting real-time environmental changes. Consequently, the raycasting module was fundamentally restructured to support dynamic, live map refreshing at every sensor cycle, pairing it with motion-based filters to prevent rotational jitter. The maximum velocity of the robot was also determined by the mapping algorithm's tolerance to function properly.

We also came across an issue with the control of the robot. The robot was supposed to be omnidirectional, however after extensive testing we came to the realisation that the problem is that the robot doesn't accept multiple velocity commands at once. This cost us precious time. Because of this limitation of the robot the controller had to be redesigned. A new constraint was added in the case of wrong alignment of the robot, where it was set to perform an in-place rotation, preventing most misalignments on the map. Although the situation was improved, sometimes during testing the robot ended up inside the inflated walls. Thus, a_star failed to find a valid path, even when it should have (invalid starting position). In order to fix this, we implemented a footprint cleaning procedure for the robot's base, which forces the small circular area around the robot to always be considered as free space, allowing the a_star path algorithm to find a valid path.

Another significant navigational challenge we encountered during the autonomous exploration phase was the "ghost frontiers" anomaly. Occasionally, anomalous LiDAR rays would penetrate through minor environmental imperfection and erroneously mark cells behind solid walls as free space. The frontier detection algorithm would recognize these isolated free cells as valid exploration targets. However, because a solid wall separated the robot from these cells, the A* path planner would fail to generate a valid path, leading to system stalls and infinite retry loops. To resolve this without altering the physical mapping logic, a Flood-Fill (Breadth-First Search) reachability mask was integrated directly into the Task Manager's decision-making pipeline. Before selecting a frontier, the algorithm dynamically floods the occupancy grid starting from the robot's current coordinates, propagating strictly through contiguous free space and terminating at obstacle boundaries. By applying a logical AND operation between the detected frontiers and the reachability mask, the system mathematically guarantees that the robot only commits to frontiers that possess a continuous, physically traversable path, completely eliminating stalls caused by ghost obstacles.

Building upon the robustness of the frontier selection process, we also addressed the critical issue of navigational deadlocks, which commonly manifested as infinite retry loops. During autonomous operation, if the robot physically failed to reach a designated frontier triggering a navigation timeout, the Task Manager would default to re-evaluating the grid. However, because the robot's physical position remained unchanged, the algorithm would repeatedly select the exact same unreachable frontier. To break this cyclical failure, a goal blacklisting mechanism was introduced. Whenever an active goal times out, its spatial coordinates are permanently logged into a blacklist memory structure. The frontier extraction algorithm was subsequently modified to include a spatial rejection filter where any candidate frontier located within a predefined radius of a blacklisted coordinate is automatically masked out and ignored. This proactive filtering forces the robot to abandon problematic or inaccessible regions and seamlessly redirect its exploration towards other viable areas, thereby guaranteeing continuous and uninterrupted autonomous operation.

Finally, transitioning the velocity controller from theoretical PI logic to physical execution on the omnidirectional myAGV platform introduced significant mechanical challenges. To be specific, the mecanum wheels proved to be highly susceptible to sudden slippage and friction inconsistencies. During initial integration, minor path deviations caused the PI controller to request abrupt, high-magnitude velocity corrections. This step-response behavior forced the motor drivers to draw maximum current instantly, causing the wheels to violently lose traction and spin out, which in turn severely degraded the structural integrity of the live map. Such behavior is shown in figures 3 and four.



Figures 3 &4: *live map mismatch due to excessive spinning of the wheels*

To resolve this, an extensive hardware tuning phase was conducted. The maximum permissible velocities were strictly capped, the lookahead distance was widened to allow for gradual cornering, and the proportional gains were carefully stepped down to ensure the controller requested smooth, mathematically stable outputs rather than sharp, instantaneous jumps. Even with these calibrations, the system is not perfectly immune to localized anomalies; sharp directional changes or sudden target updates as the robot approaches a goal coordinate can still induce transient wheel slipping. Within the scope of this project, this is deemed an acceptable physical limitation. The complex trade-off between maintaining a highly responsive path tracker and accommodating the low-friction physical realities of the hardware was successfully managed, ensuring the overarching autonomous objectives are reliably accomplished despite minor mechanical slips. The results of the complete mapping visible in rviz are shown in figure 5.

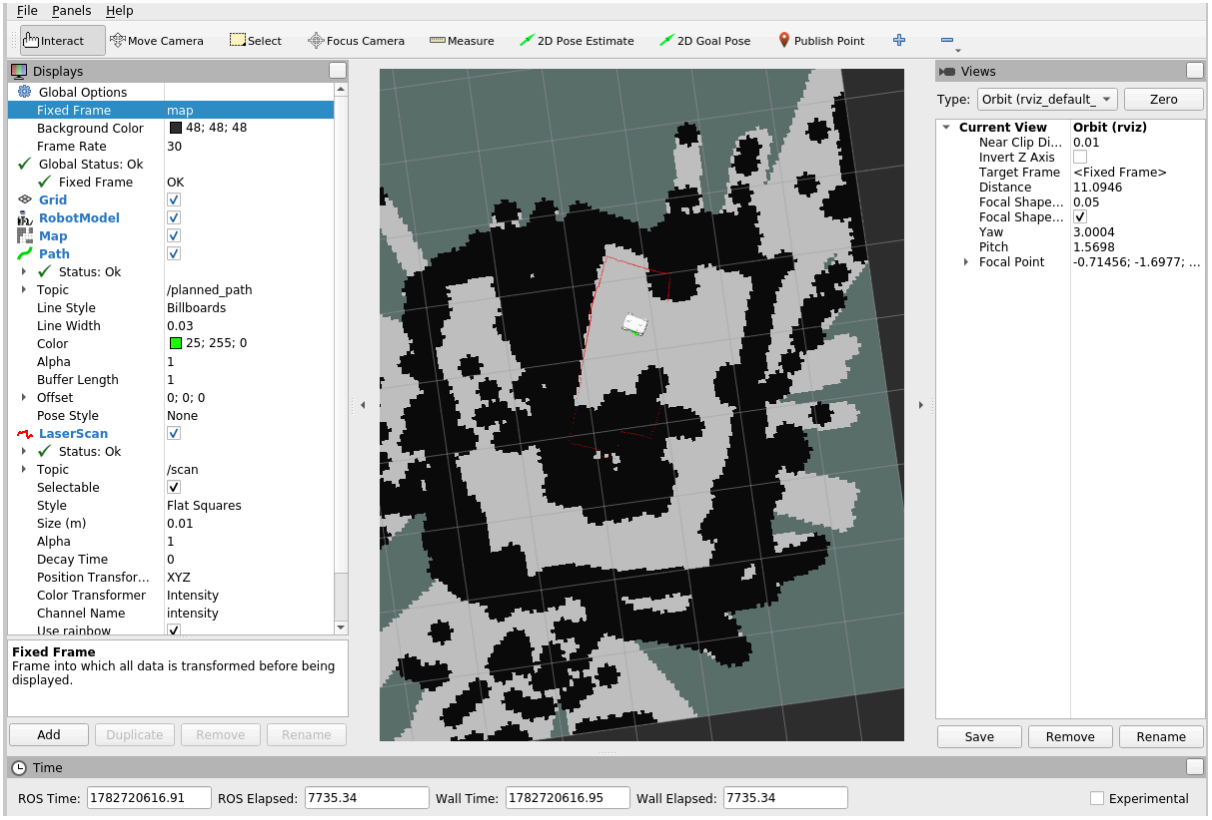


Figure 5: *Complete visualization of the map*

Lastly, we created a launch file for RViz to keep the configurations saved. We also made a launch file for all the working nodes (our custom ones and the robot_state_publisher), including the vision_node and the watchdog, even if they are not fully functional, to present the full build of the project. For execution purposes, the files are launched individually as stated in the read_me file.

References

Elephant Robotics. (2023). *MyAGV 2023 Pi Official Documentation & ROS2 Packages*. Retrieved from https://github.com/elephantrobotics/myagv_ros2

Besl, P. J., & McKay, N. D. (1992). A method for registration of 3-D shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 14*(2), 239-256.

Bai, H. (2022). *ICP Algorithm: Theory, Practice And Its SLAM-oriented Taxonomy*. University of Illinois Urbana-Champaign (UIUC).

Arun, K. S., Huang, T. S., & Blostein, S. D. (1987). Least-squares fitting of two 3-D point sets. *IEEE Transactions on Pattern Analysis and Machine Intelligence, (5)*, 698-700.

Bresenham, J. E. (1965). Algorithm for computer control of a digital plotter. *IBM Systems Journal, 4*(1), 25-30.

Patel, A. (2020). *Heuristics for Grid Paths*. Stanford University. Retrieved from http://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html

Garrido-Jurado, S., Muñoz-Salinas, R., Madrid-Cuevas, F. J., & Marín-Jiménez, M. J. (2014). Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition, 47*(6), 2280-2292.

https://www.geeksforgeeks.org/electronics-engineering/proportional-integral-controller-control-system/